# External RPC API

> **Danger**
>
> > **Note**
> >
> > 19.0
>
> Both the XML-RPC and JSON-RPC APIs at endpoints `/xmlrpc`, `/xmlrpc/2` and `/jsonrpc` are scheduled for removal in Odoo 22 (fall 2028) and Online 21.1 (winter 2027). The **External JSON-2 API** acts as a replacement.
>
> The other controllers `@route(type='jsonrpc')` (known until Odoo 18 as `type='json'`) are not subject to this deprecation notice.

Odoo is usually extended internally via modules, but many of its features and all of its data are also available from the outside for external analysis or integration with various tools. Part of the **Models** API is easily available over **XML-RPC** and accessible from a variety of languages.

Starting with PHP8, the XML-RPC extension may not be available by default. Check out the **manual** for the installation steps.

Access to data via the external API is only available on *Custom* Odoo pricing plans. Access to the external API is not available on *One App Free* or *Standard* plans. For more information visit the **Odoo pricing page** or reach out to your Customer Success Manager.

## Connection

### Configuration

If you already have an Odoo server installed, you can just use its parameters.

> **Important**
>
> For Odoo Online instances (<domain>.odoo.com), users are created without a *local* password (as a person you are logged in via the Odoo Online authentication system, not

odoo docs

- Go to **Settings ▸ Users & Companies ▸ Users**.
- Click on the user you want to use for XML-RPC access.
- Click on **Action** and select **Change Password**.
- Set a **New Password** value then click **Change Password**.

The *server url* is the instance's domain (e.g. *https://mycompany.odoo.com*), the *database name* is the name of the instance (e.g. *mycompany*). The *username* is the configured user's login as shown by the *Change Password* screen.

**Python** | Ruby | PHP | Java | Go

```
url = <insert server URL>
db = <insert database name>
username = 'admin'
password = <insert password for your admin user (default: a
```

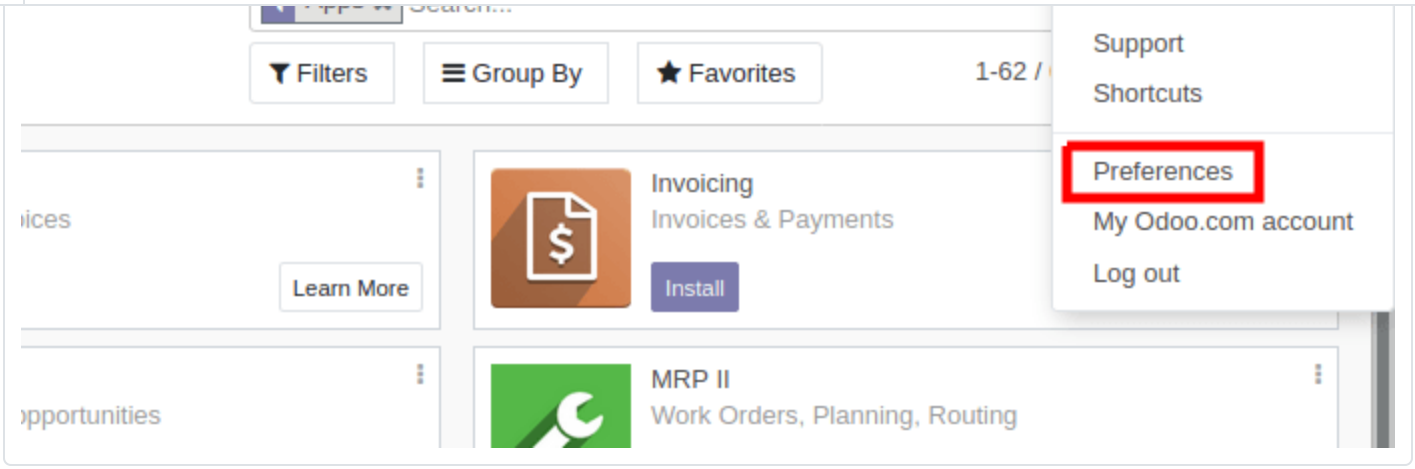
## API Keys

*New in version 14.0.*

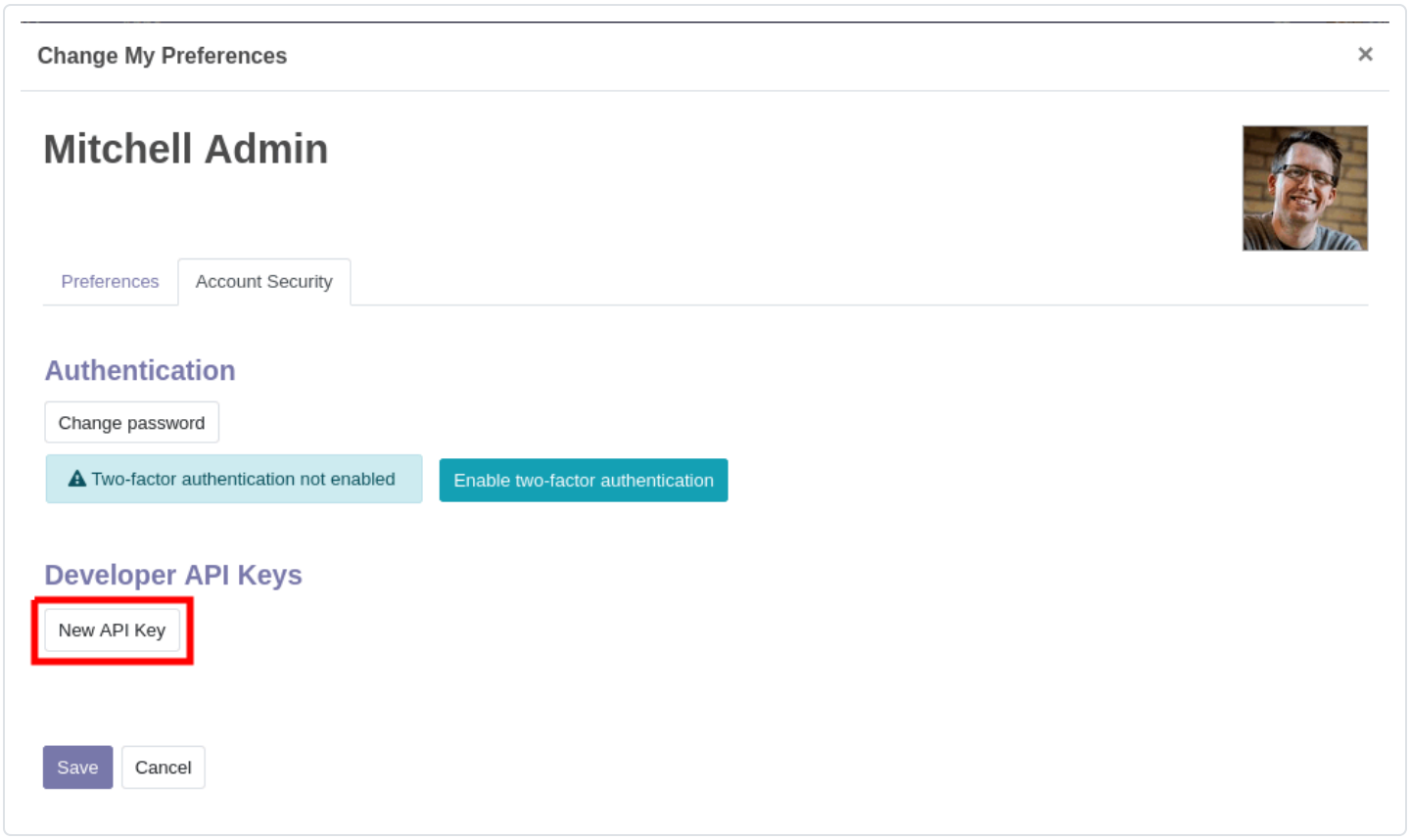
Odoo has support for **api keys** and (depending on modules or settings) may **require** these keys to perform webservice operations.

The way to use API Keys in your scripts is to simply replace your **password** by the key. The login remains in-use. You should store the API Key as carefully as the password as they essentially provide the same access to your user account (although they can not be used to log-in via the interface).

In order to add a key to your account, simply go to your **Preferences** (or **My Profile**):



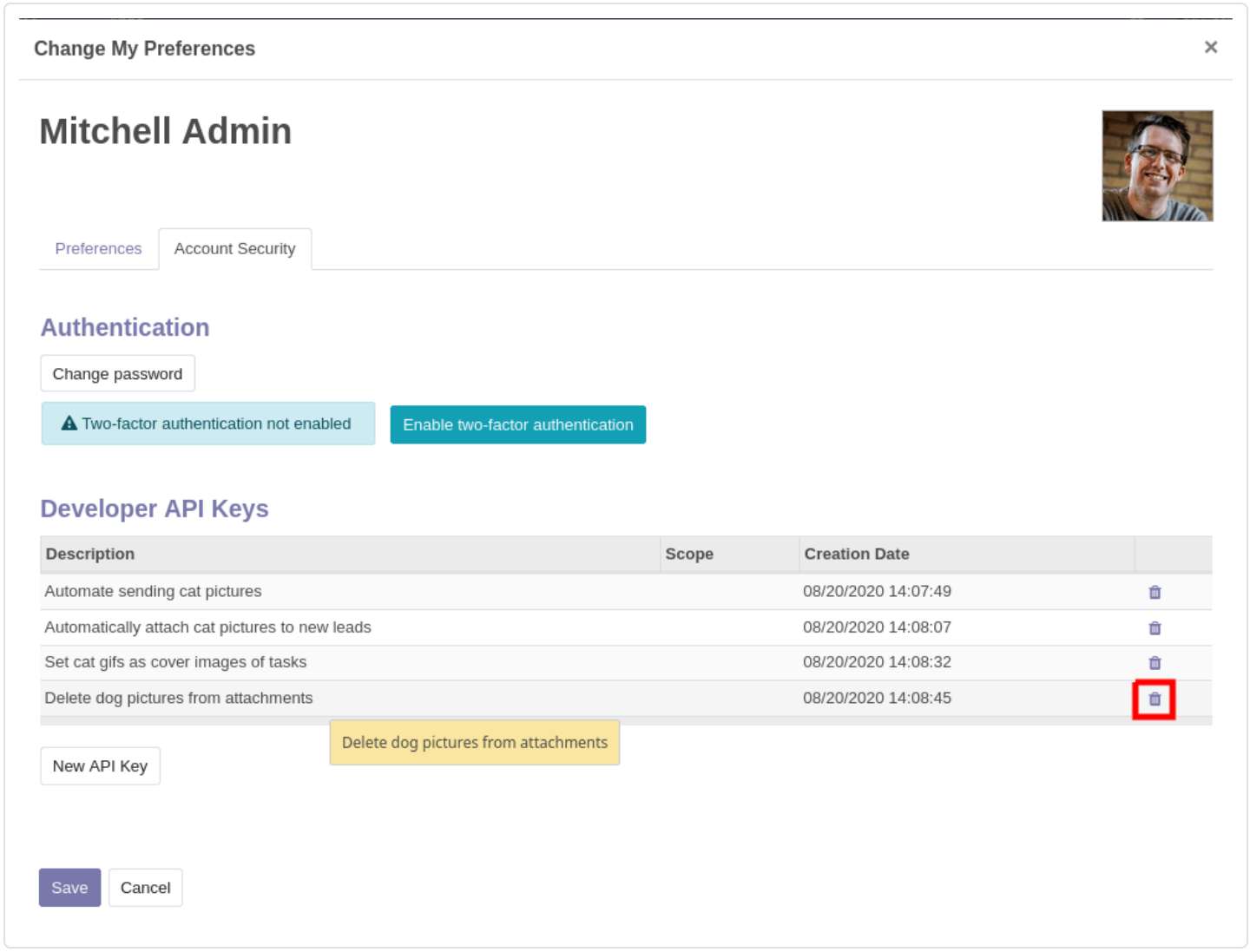

then open the **Account Security** tab, and click **New API Key**:




Input a description for the key, **this description should be as clear and complete as possible**: it is the only way you will have to identify your keys later and know whether you should remove them or keep them around.

Click **Generate Key**, then copy the key provided. **Store this key carefully**: it is equivalent to your password, and just like your password the system will not be able to retrieve or show the

button, and you will be able to delete them:




**A deleted API key can not be undeleted or re-set**. You will have to generate a new key and update all the places where you used the old one.

## Test database

To make exploration simpler, you can also ask **https://demo.odoo.com** for a test database:

Python | Ruby | PHP | Java | Go

```
import xmlrpc.client
info = xmlrpc.client.ServerProxy('https://demo.odoo.com/sta
```

odoo docs

## Logging in

Odoo requires users of the API to be authenticated before they can query most data.

The `xmlrpc/2/common` endpoint provides meta-calls which don't require authentication, such as the authentication itself or fetching version information. To verify if the connection information is correct before trying to authenticate, the simplest call is to ask for the server's version. The authentication itself is done through the `authenticate` function and returns a user identifier ( `uid` ) used in authenticated calls instead of the login.

**Python** Ruby PHP Java Go

```
common = xmlrpc.client.ServerProxy('{}/xmlrpc/2/common'.fo
common.version()
```

Result:

```
{
    "server_version": "13.0",
    "server_version_info": [13, 0, 0, "final", 0],
    "server_serie": "13.0",
    "protocol_version": 1,
}
```

**Python** Ruby PHP Java Go

```
uid = common.authenticate(db, username, password, {})
```

# Calling methods

The second endpoint is `xmlrpc/2/object` . It is used to call methods of odoo models via the `execute_kw` RPC function.

- the user's password, a string
- the model name, a string
- the method name, a string
- an array/list of parameters passed by position
- a mapping/dict of parameters to pass by keyword (optional)

### Example

For instance, to search for records in the `res.partner` model, we can call `name_search` with `name` passed by position and `limit` passed by keyword (in order to get maximum 10 results):

**Python** | Ruby | PHP | Java | Go

```
models = xmlrpc.client.ServerProxy('{}/xmlrpc/2/object'.fo
models.execute_kw(db, uid, password, 'res.partner', 'name_
```

Result:

```
true
```

## List records

Records can be listed and filtered via `search()`.

`search()` takes a mandatory **domain** filter (possibly empty), and returns the database identifiers of all records matching the filter.

### Example

To list customer companies, for instance:

**Python** | Ruby | PHP | Java | Go

odoo docs

Result:

```
[7, 18, 12, 14, 17, 19, 8, 31, 26, 16, 13, 20, 30, 22, 29, 15, 23, 28, 74]
```

## Pagination

By default a search will return the ids of all records matching the condition, which may be a huge number. `offset` and `limit` parameters are available to only retrieve a subset of all matched records.

### Example

**Python** | Ruby | PHP | Java | Go

```
models.execute_kw(db, uid, password, 'res.partner', 'searcl
```

Result:

```
[13, 20, 30, 22, 29]
```

## Count records

Rather than retrieve a possibly gigantic list of records and count them, `search_count()` can be used to retrieve only the number of records matching the query. It takes the same **domain** filter as `search()` and no other parameter.

### Example

**Python** | Ruby | PHP | Java | Go

Result:

```
19
```

**Note**

Calling `search` then `search_count` (or the other way around) may not yield coherent results if other users are using the server: stored data could have changed between the calls.

## Read records

Record data are accessible via the `read()` method, which takes a list of ids (as returned by `search()`), and optionally a list of fields to fetch. By default, it fetches all the fields the current user can read, which tends to be a huge amount.

**Example**

**Python** | Ruby | PHP | Java | Go

```
ids = models.execute_kw(db, uid, password, 'res.partner',
[record] = models.execute_kw(db, uid, password, 'res.partn
# count the number of fields fetched by default
len(record)
```

Result:

```
121
```

Conversely, picking only three fields deemed interesting.

**Python** | Ruby | PHP | Java | Go

odoo docs

Result:

```
[{"comment": false, "country_id": [21, "Belgium"], "id": 7, "name": "Agrolait"}
```

**Note**

Even if the `id` field is not requested, it is always returned.

## List record fields

`fields_get()` can be used to inspect a model's fields and check which ones seem to be of interest.

Because it returns a large amount of meta-information (it is also used by client programs) it should be filtered before printing, the most interesting items for a human user are `string` (the field's label), `help` (a help text if available) and `type` (to know which values to expect, or to send when updating a record).

**Example**

Python | Ruby | PHP | Java | Go

```
models.execute_kw(db, uid, password, 'res.partner', 'fields
```

Result:

```
{
    "ean13": {
        "type": "char",
        "help": "BarCode",
        "string": "EAN13"
    },
    "property_account_position_id": {
```

```
    "signup_valid": {
        "type": "boolean",
        "help": "",
        "string": "Signup Token is Valid"
    },
    "date_localization": {
        "type": "date",
        "help": "",
        "string": "Geo Localization Date"
    },
    "ref_company_ids": {
        "type": "one2many",
        "help": "",
        "string": "Companies that refers to partner"
    },
    "sale_order_count": {
        "type": "integer",
        "help": "",
        "string": "# of Sales Order"
    },
    "purchase_order_count": {
        "type": "integer",
        "help": "",
        "string": "# of Purchase Order"
    },
```


## Search and read

Because it is a very common task, Odoo provides a `search_read()` shortcut which, as its name suggests, is equivalent to a `search()` followed by a `read()`, but avoids having to perform two requests and keep ids around.

Its arguments are similar to `search()`'s, but it can also take a list of `fields` (like `read()`, if that list is not provided it will fetch all fields of matched records).

### Example

**Python** | Ruby | PHP | Java | Go

odoo docs

Result:


```
[
    {
        "comment": false,
        "country_id": [ 21, "Belgium" ],
        "id": 7,
        "name": "Agrolait"
    },
    {
        "comment": false,
        "country_id": [ 76, "France" ],
        "id": 18,
        "name": "Axelor"
    },
    {
        "comment": false,
        "country_id": [ 233, "United Kingdom" ],
        "id": 12,
        "name": "Bank Wealthy and sons"
    },
    {
        "comment": false,
        "country_id": [ 105, "India" ],
        "id": 14,
        "name": "Best Designers"
    },
    {
        "comment": false,
        "country_id": [ 76, "France" ],
        "id": 17,
        "name": "Camptocamp"
    }
]
```


## Create records

Records of a model are created using `create()`. The method creates a single record and returns its database identifier.

odoo docs

### Example

Python | Ruby | PHP | Java | Go

```
id = models.execute_kw(db, uid, password, 'res.partner', '
```

Result:

```
78
```

### Warning

While most value types are what would expect (integer for `Integer`, string for `Char` or `Text`),

- `Date`, `Datetime` and `Binary` fields use string values
- `One2many` and `Many2many` use a special command protocol detailed in `the documentation to the write method`.

## Update records

Records can be updated using `write()`. It takes a list of records to update and a mapping of updated fields to values similar to `create()`.

Multiple records can be updated simultaneously, but they will all get the same values for the fields being set. It is not possible to perform "computed" updates (where the value being set depends on an existing value of a record).

### Example

Python | Ruby | PHP | Java | Go

```
models.execute_kw(db, uid, password, 'res.partner', 'write
# get record name after having changed it
```

odoo docs

Result:

```
[[78, "Newer partner"]]
```

## Delete records

Records can be deleted in bulk by providing their ids to `unlink()` .

### Example

Python | Ruby | PHP | Java | Go

```
models.execute_kw(db, uid, password, 'res.partner', 'unlinl
# check if the deleted record is still in the database
models.execute_kw(db, uid, password, 'res.partner', 'searcl
```

Result:

```
[]
```

## Inspection and introspection

While we previously used `fields_get()` to query a model and have been using an arbitrary model from the start, Odoo stores most model metadata inside a few meta-models which allow both querying the system and altering models and fields (with some limitations) on the fly over XML-RPC.

### `ir.model`

Provides information about Odoo models via its various fields.

**`name`**
a human-readable description of the model

**`model`**

odoo docs

`manual` )

**`field_id`**
list of the model's fields through a `One2many` to **ir.model.fields**

**`view_ids`**
`One2many` to the **View architectures** defined for the model

**`access_ids`**
`One2many` relation to the **Access Rights** set on the model

`ir.model` can be used to

- Query the system for installed models (as a precondition to operations on the model or to explore the system's content).
- Get information about a specific model (generally by listing the fields associated with it).
- Create new models dynamically over RPC.

### Important

- Custom model names must start with `x_`.
- The `state` must be provided and set to `manual`, otherwise the model will not be loaded.
- It is not possible to add new *methods* to a custom model, only fields.

### Example

A custom model will initially contain only the "built-in" fields available on all models:

**Python** | PHP | Ruby | Java | Go

```
models.execute_kw(db, uid, password, 'ir.model', 'create',
    'name': "Custom Model",
    'model': "x_custom_model",
    'state': 'manual',
}])
models.execute_kw(db, uid, password, 'x_custom_model', 'fi
```

Result:

```
        "string": "Created by"
    },
    "create_date": {
        "type": "datetime",
        "string": "Created on"
    },
    "__last_update": {
        "type": "datetime",
        "string": "Last Modified on"
    },
    "write_uid": {
        "type": "many2one",
        "string": "Last Updated by"
    },
    "write_date": {
        "type": "datetime",
        "string": "Last Updated on"
    },
    "display_name": {
        "type": "char",
        "string": "Display Name"
    },
    "id": {
        "type": "integer",
        "string": "Id"
    }
}
```


### `ir.model.fields`

Provides information about the fields of Odoo models and allows adding custom fields without using Python code.

**`model_id`**
`Many2one` to ir.model to which the field belongs

**`name`**
the field's technical name (used in `read` or `write`)

**`field_description`**
the field's user-readable label (e.g. `string` in `fields_get`)

odoo docs

whether the field was created via Python code ( `base` ) or via `ir.model.fields` ( `manual` )

**`required` , `readonly` , `translate`**
enables the corresponding flag on the field

**`groups`**
field-level access control, a **`Many2many`** to `res.groups`

**`selection` , `size` , `on_delete` , `relation` , `relation_field` , `domain`**
type-specific properties and customizations, see the fields documentation for details

### Important

- Like custom models, only new fields created with `state="manual"` are activated as actual fields on the model.
- Computed fields can not be added via `ir.model.fields`, some field meta-information (defaults, onchange) can not be set either.

### Example

**Python** | PHP | Ruby | Java | Go

```
id = models.execute_kw(db, uid, password, 'ir.model', 'crea
    'name': "Custom Model",
    'model': "x_custom",
    'state': 'manual',
}])
models.execute_kw(db, uid, password, 'ir.model.fields', 'cr
    'model_id': id,
    'name': 'x_name',
    'ttype': 'char',
    'state': 'manual',
    'required': True,
}])
record_id = models.execute_kw(db, uid, password, 'x_custom
models.execute_kw(db, uid, password, 'x_custom', 'read', [
```

Result:

```
        "x_name": "test record",
        "__last_update": "2014-11-12 16:32:13",
        "write_uid": [1, "Administrator"],
        "write_date": "2014-11-12 16:32:13",
        "create_date": "2014-11-12 16:32:13",
        "id": 1,
        "display_name": "test record"
    }
]
```


## Get Help

Contact Support | Ask the Odoo Community